

LAB ACTION PLAN FOR WEEK 6

Objectives:

Students will be able to:

- Learn how to define and run multiple interdependent services (e.g., web server, database) in a single configuration file.
- Gain skills in writing and interpreting docker-compose.yml files for service setup.
- Deploy the same setup across different machines without manual configuration.
- Configure container networking and persistent storage within Compose.
- Reduce setup time and enable faster iteration during application development.

Students will perform Hands-on running multi container applications using docker compose and make document that include the execution of each step along with scenario-based question solutions after completing the tasks.

What is Docker Compose?

Docker Compose is a tool used to define and run multi-container Docker applications. It allows you to define services, networks, and volumes that your application needs, all in a single file. This makes it easier to manage complex applications that require multiple containers (e.g., a web server and a database).

Structure of a Docker Compose File

A Docker Compose file is written in YAML (YAML Ain't Markup Language), a human-readable data format used for configuration. In the Docker context, it helps define the services, networks, and volumes needed for an application.

Basic Components of a Docker Compose YAML File

1. **Version:** Specifies which version of the Docker Compose file format you're using. Each version comes with its own set of features.

version: '3.8'

2. **Services:** This section defines all the containers you want to run for your application. Each service represents a container (like a web server or database).

Each service includes:

- **Image:** The Docker image to use.
- **Ports:** Ports to map between the container and your host machine.
- **Environment:** Environment variables required by the service.
- **Dependencies:** Which other services a container depends on.

Example:

```
services:
  web:
    image: nginx
    ports:
      - "8060:80"
  db:
    image: tomeet
    ports:
      - "8050:8080"
```

3. **Networks:** Define networks to allow different services to communicate with each other. Docker Compose automatically creates a default network if not specified.
4. **Volumes:** Used for persistent storage, allowing data to persist even after the container stops or is removed.

Example Docker Compose File (Simple)

Here's a basic example of a Docker Compose file that runs WordPress and MySQL together:

```
version: '3.8' # Docker Compose file format version

services:
  wordpress: # WordPress service
    image: wordpress:latest
    ports:
      - "8080:80" # Map port 80 of the container to port 8080 of the host
    environment:
      WORDPRESS_DB_HOST: db:3306 # Database host
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress
      WORDPRESS_DB_NAME: wordpress
    depends_on:
      - db # Ensures the db service starts first

  db: # MySQL service
    image: mysql:5.7
```

environment:
MYSQL_ROOT_PASSWORD: rootpassword
MYSQL_DATABASE: wordpress
MYSQL_USER: wordpress
MYSQL_PASSWORD: wordpress

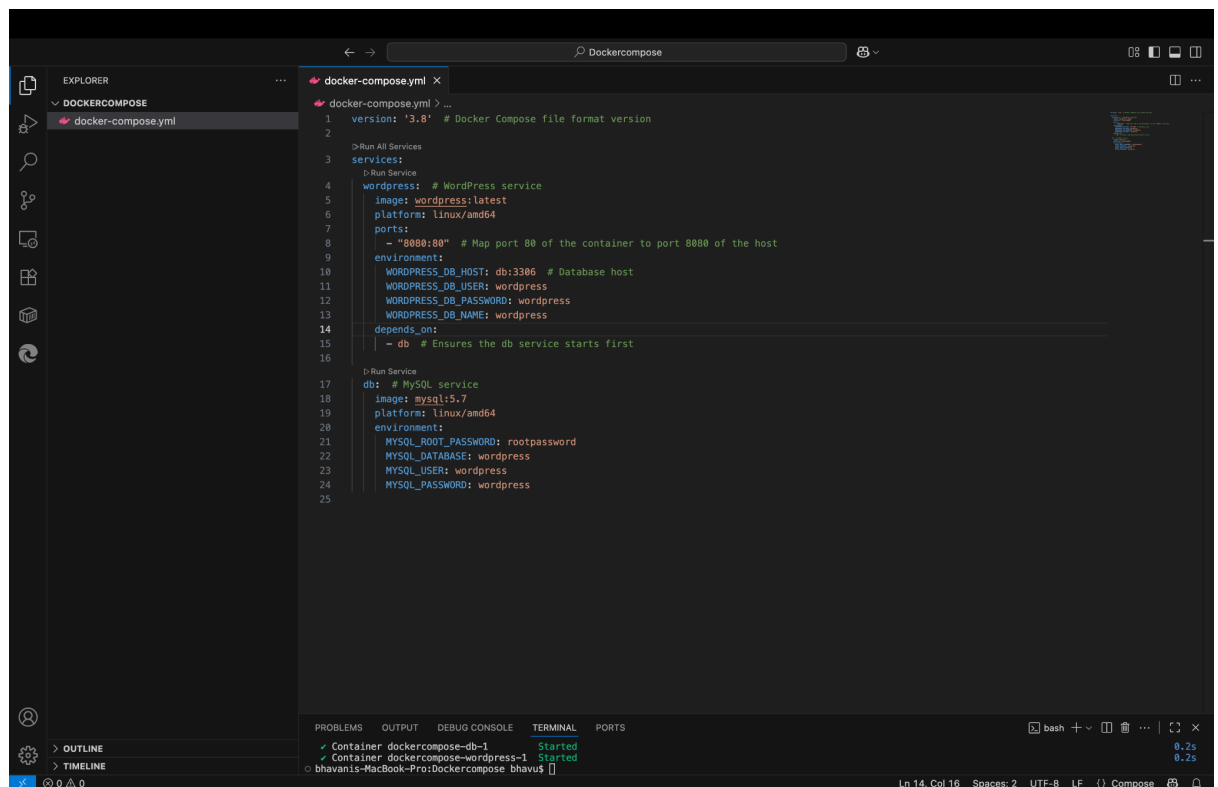
How to Run It

To run a multi-container setup like the one above:

1. Save the file as docker-compose.yml.

Or

Docker-compose.yaml

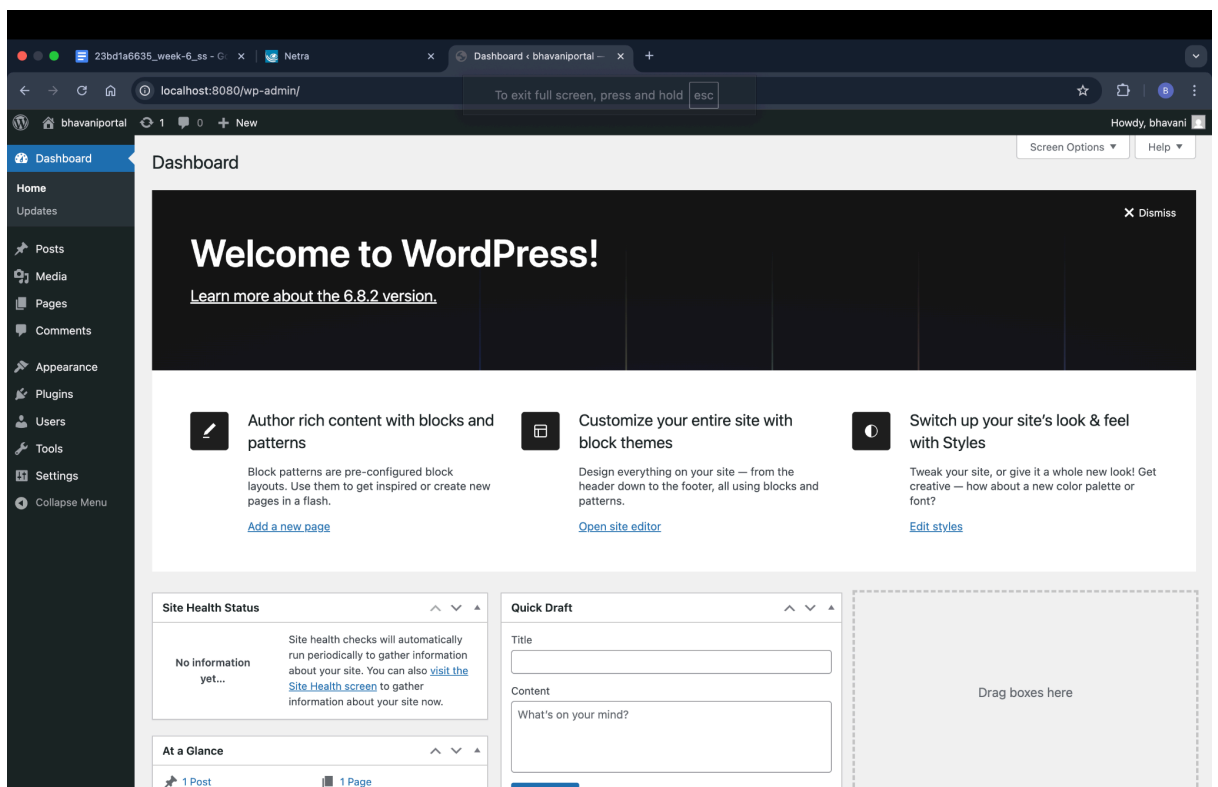


2. To Start the compose

docker-compose up -d

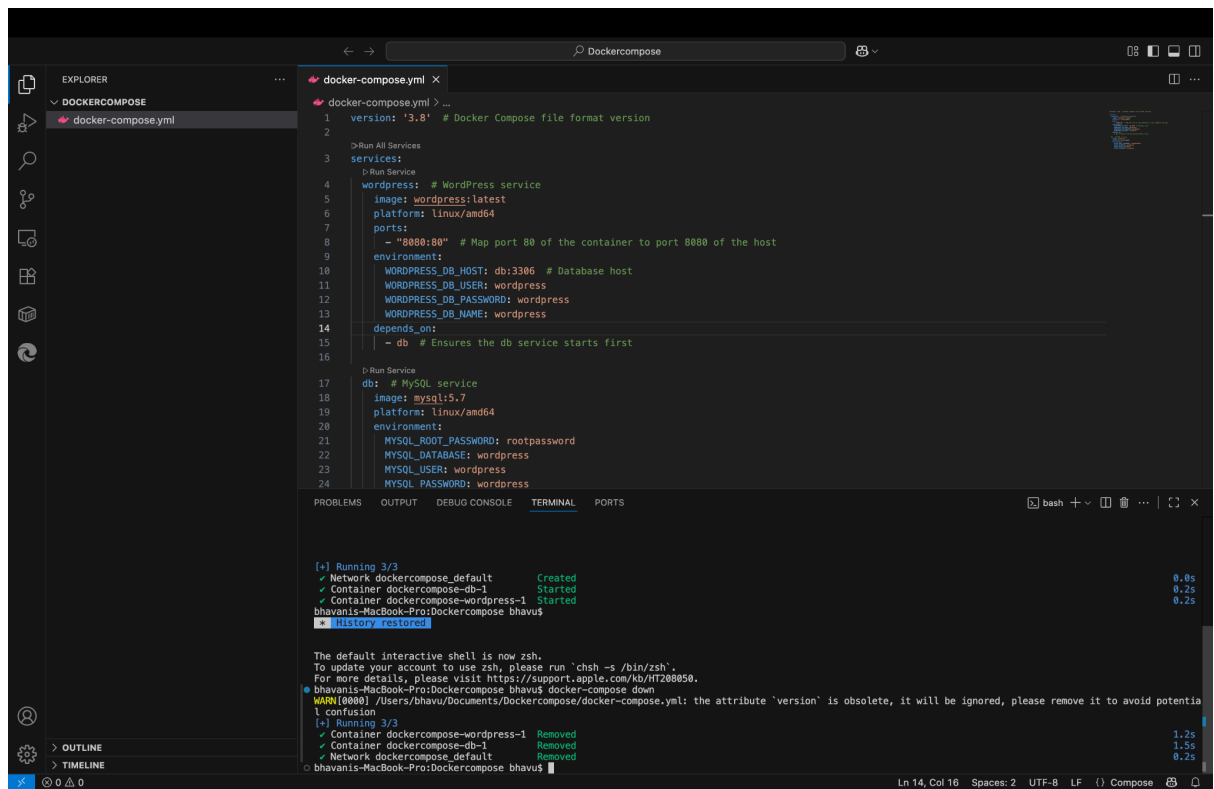
```
docker-compose.yml
1 version: '3.8' # Docker Compose file format version
2
3 >Run All Services
4 services:
5   >Run Service
6   wordpress: # WordPress service
7     image: wordpress:latest
8     platform: linux/amd64
9     ports:
10      - "8080:80" # Map port 80 of the container to port 8080 of the host
11     environment:
12       WORDPRESS_DB_HOST: db:3306 # Database host
13       WORDPRESS_DB_USER: wordpress
14       WORDPRESS_DB_PASSWORD: wordpress
15       WORDPRESS_DB_NAME: wordpress
16     depends_on:
17       - db # Ensure the db service starts first
```

```
bash
The default interactive shell is now zsh.
To update your account to use zsh, please run 'chsh -s /bin/zsh'.
For more details, please visit https://support.apple.com/kb/HT208050.
bhavanis-MacBook-Pro:Dockercompose bhavus$ docker-compose up -d
WARN[0000] /Users/bhavu/Documents/Dockercompose/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 3/3
  ✓ Network dockercompose_default      Created      0.0s
  ✓ Container dockercompose-db-1       Started      0.2s
  ✓ Container dockercompose-wordpress-1 Started      0.2s
bhavanis-MacBook-Pro:Dockercompose bhavus$
```



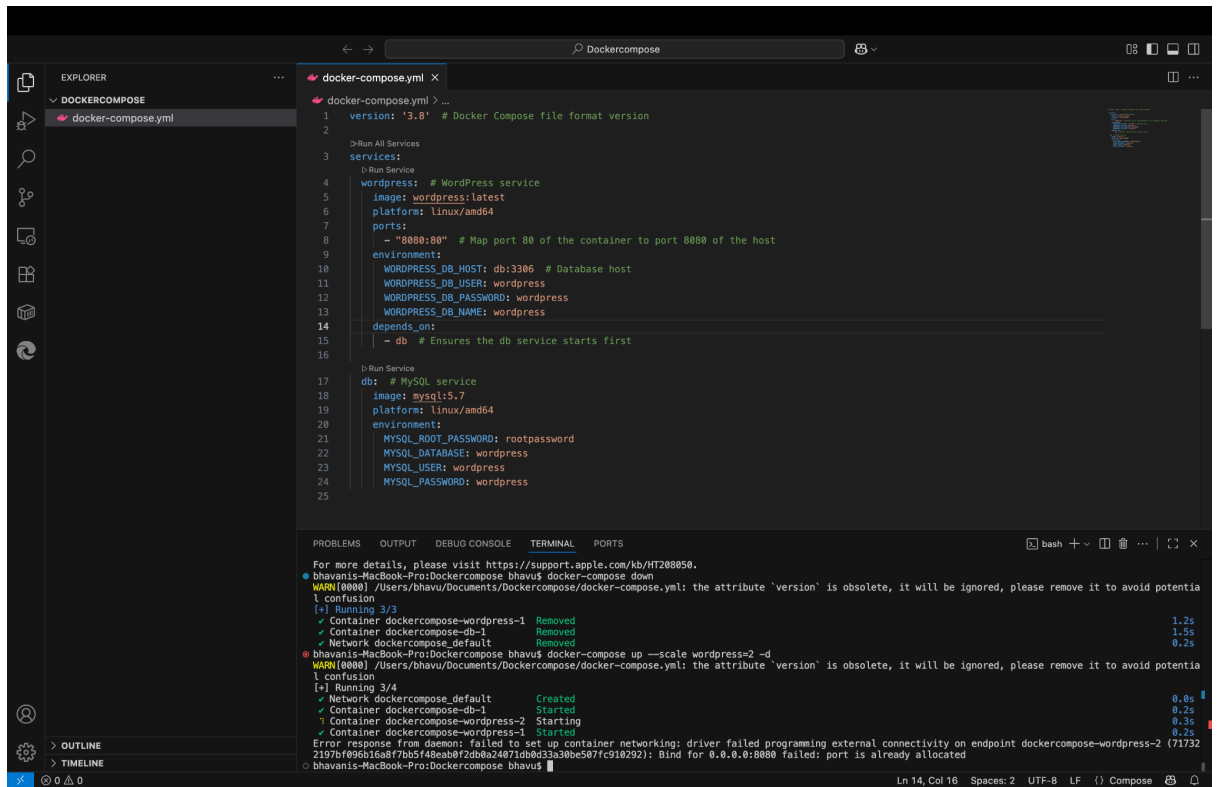
3. To stop the containers

docker-compose down



4. To scale the container

`docker-compose up --scale <service name>=2 -d`



1. Define and run multiple interdependent services

Task:

I. Create a new folder compose-lab

Inside it, create a file docker-compose.yml with the following content:

version: "3.9"

services:

web:

image: nginx:latest

ports:

- "8080:80"

db:

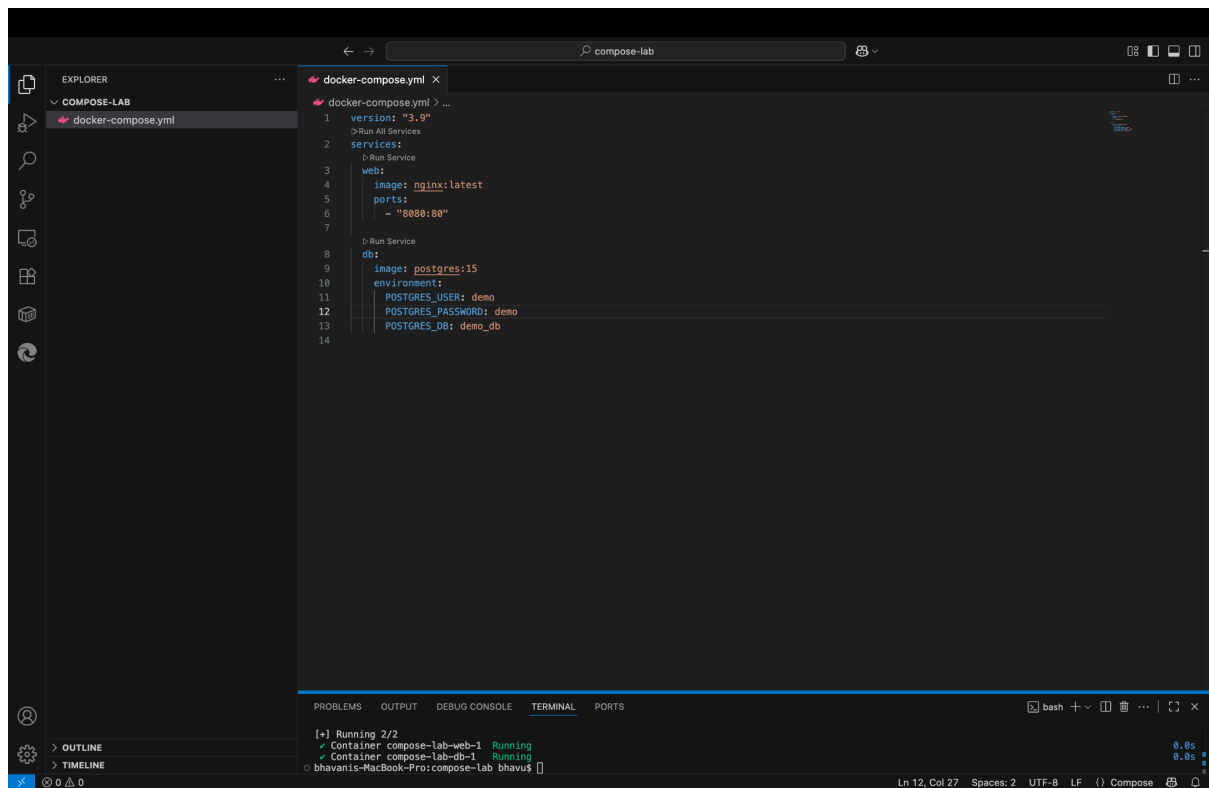
image: postgres:15

environment:

POSTGRES_USER: demo

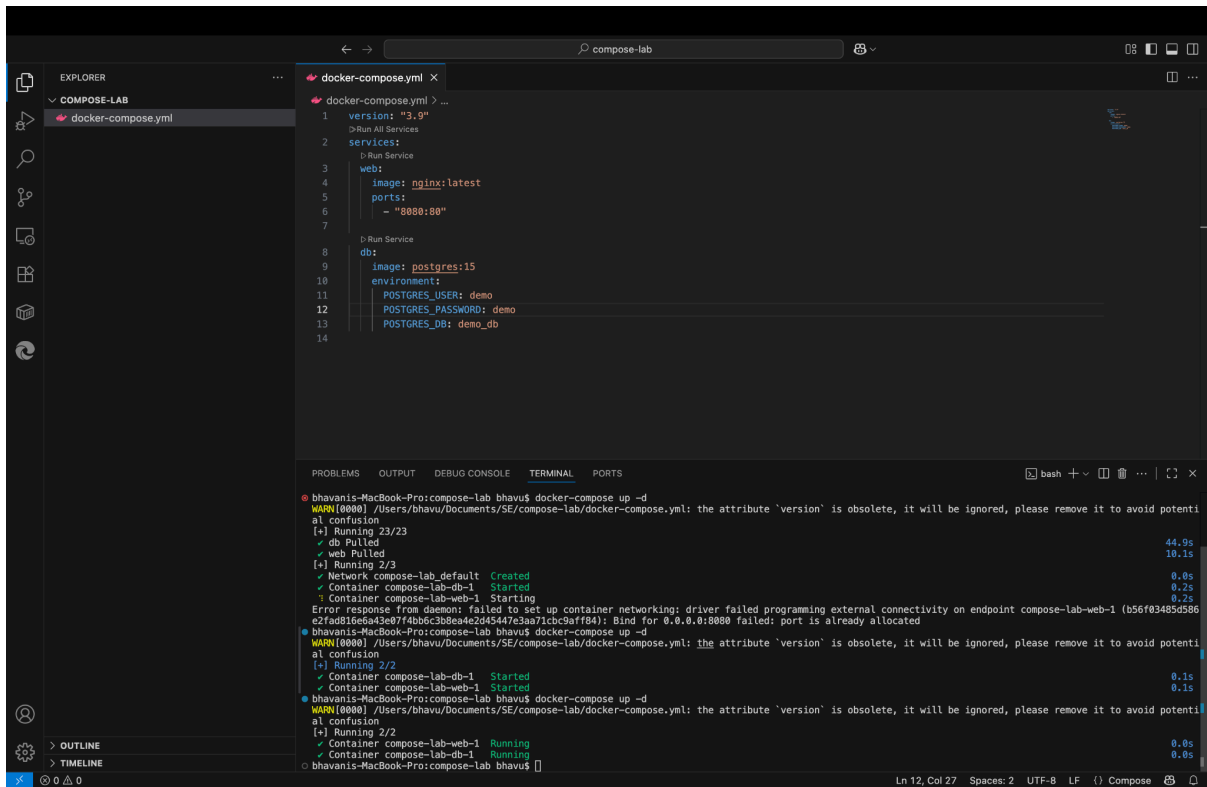
POSTGRES_PASSWORD: demo

POSTGRES_DB: demo_db



II. Run the setup:

docker compose up -d

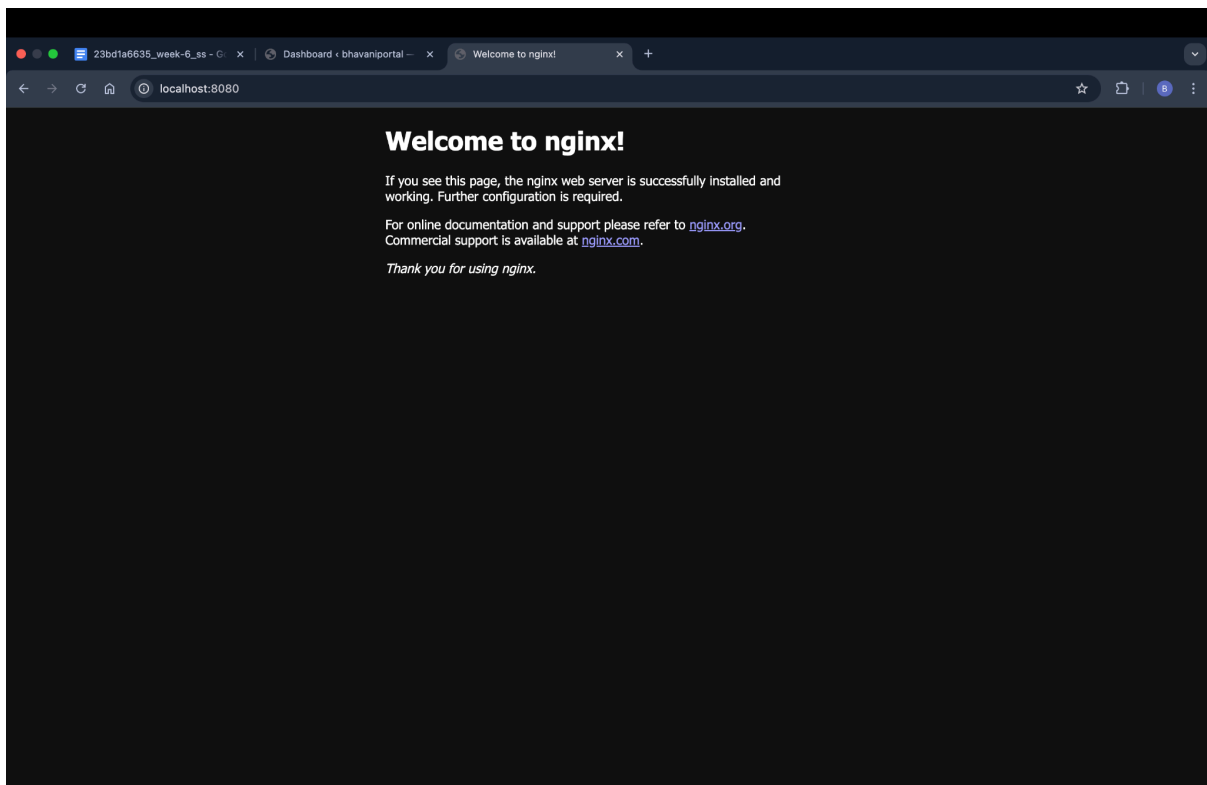


The screenshot shows a VS Code editor with a file named `docker-compose.yml` open. The file contains the following configuration:

```
1 version: '3.9'  
2  
3 services:  
4   web:  
5     image: nginx:latest  
6     ports:  
7       - "8080:80"  
8  
9   db:  
10    image: postgres:15  
11    environment:  
12      POSTGRES_USER: demo  
13      POSTGRES_PASSWORD: demo  
14      POSTGRES_DB: demo_db
```

The terminal at the bottom shows the output of the command `docker-compose up -d`. It displays a warning about the obsolete `version` attribute, followed by the successful creation and starting of the `web` and `db` containers. The output also shows the network `compose-lab_default` being created and the containers being started.

III. Open your browser and visit: <http://localhost:8080>.



IV. Expected Output:

Nginx welcome page is displayed.

db container runs in the background.

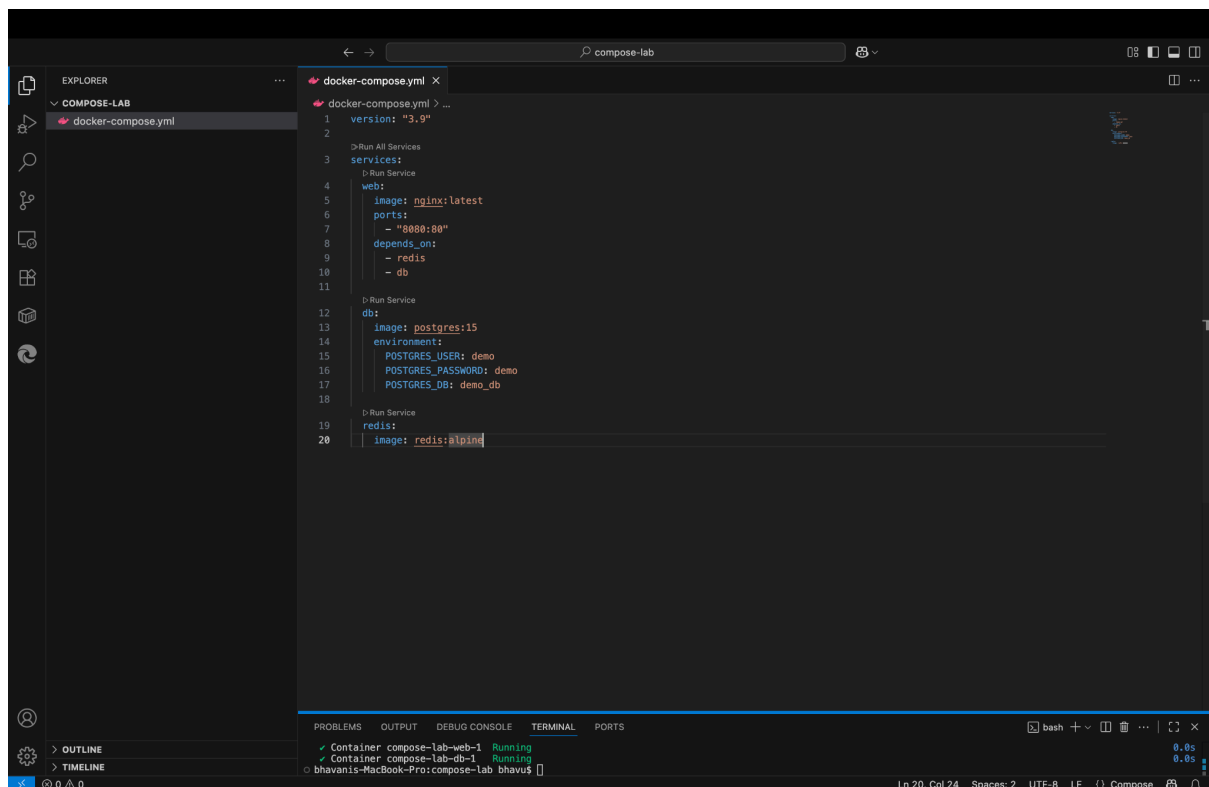
2. Write and interpret docker-compose.yml files

Task:

I. Modify docker-compose.yml to add a Redis cache:

redis:

image: redis:alpine



II. Add a depends_on so web waits for Redis:

web:

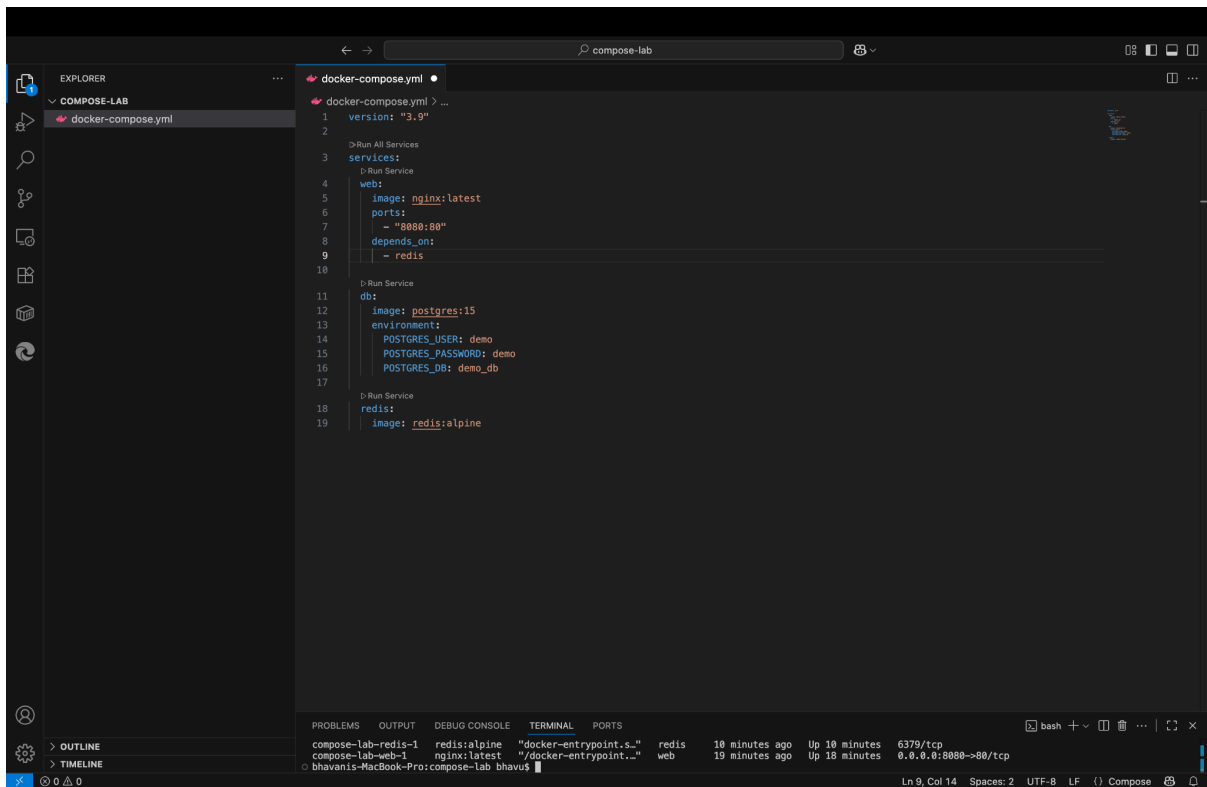
image: nginx:latest

ports:

- "8080:80"

depends_on:

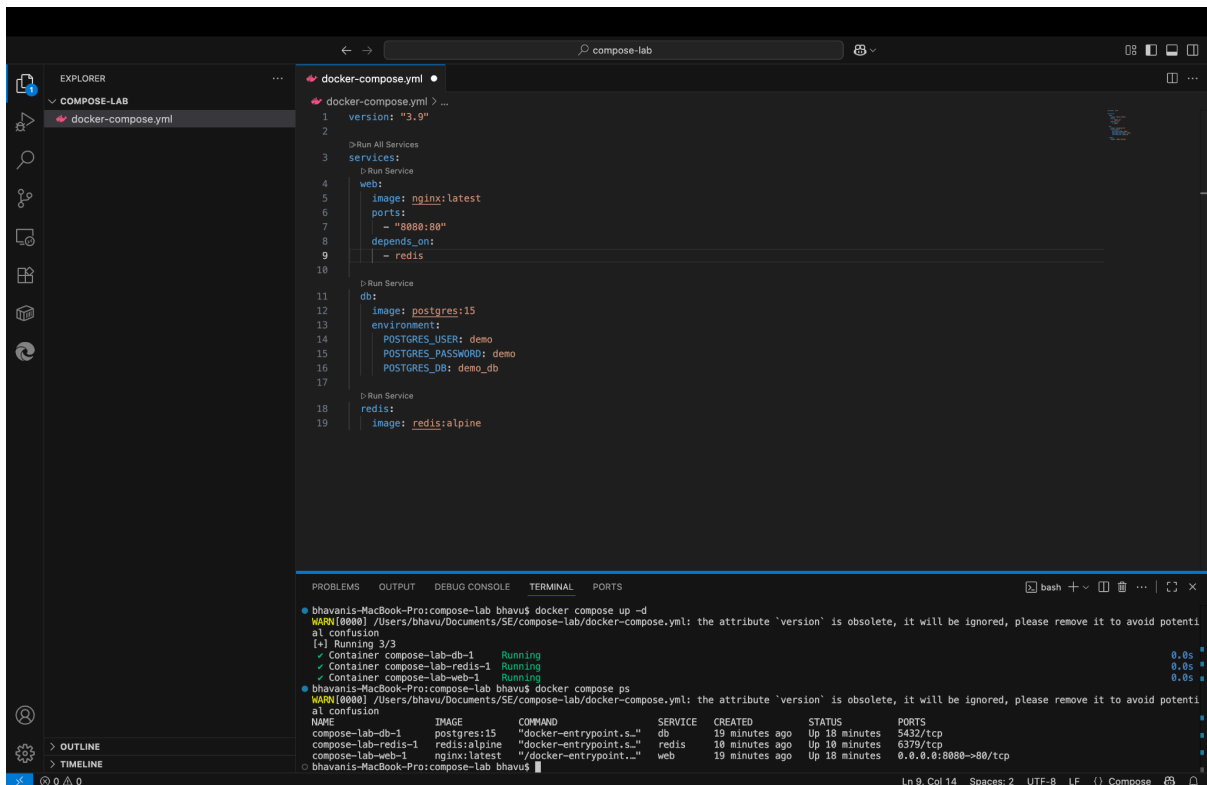
- redis



III. Restart the setup:

docker compose up -d

docker compose ps



IV. Expected Output:

Three services (web, db, redis) are listed as running.

```
Container compose-lab-web-1 Running 0.0s
bhavanis-MacBook-Pro:compose-lab bhavus$ docker compose ps
WARN[0000] /Users/bhavu/Documents/SE/compose-lab/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potenti
al confusion
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
compose-lab-db-1    postgres:15         "docker-entrypoint.s..." db         9 minutes ago  Up 9 minutes  5432/tcp
compose-lab-redis-1 redis:alpine         "docker-entrypoint.s..." redis      21 seconds ago Up 20 seconds  6379/tcp
compose-lab-web-1   nginx:latest        "/docker-entrypoint..." web        9 minutes ago  Up 9 minutes  0.0.0.0:8080->80/tcp
bhavanis-MacBook-Pro:compose-lab bhavus$
```

3. Deploy across different machines

Task:

I. Zip your compose-lab folder.

Transfer it to another machine with Docker Compose installed.

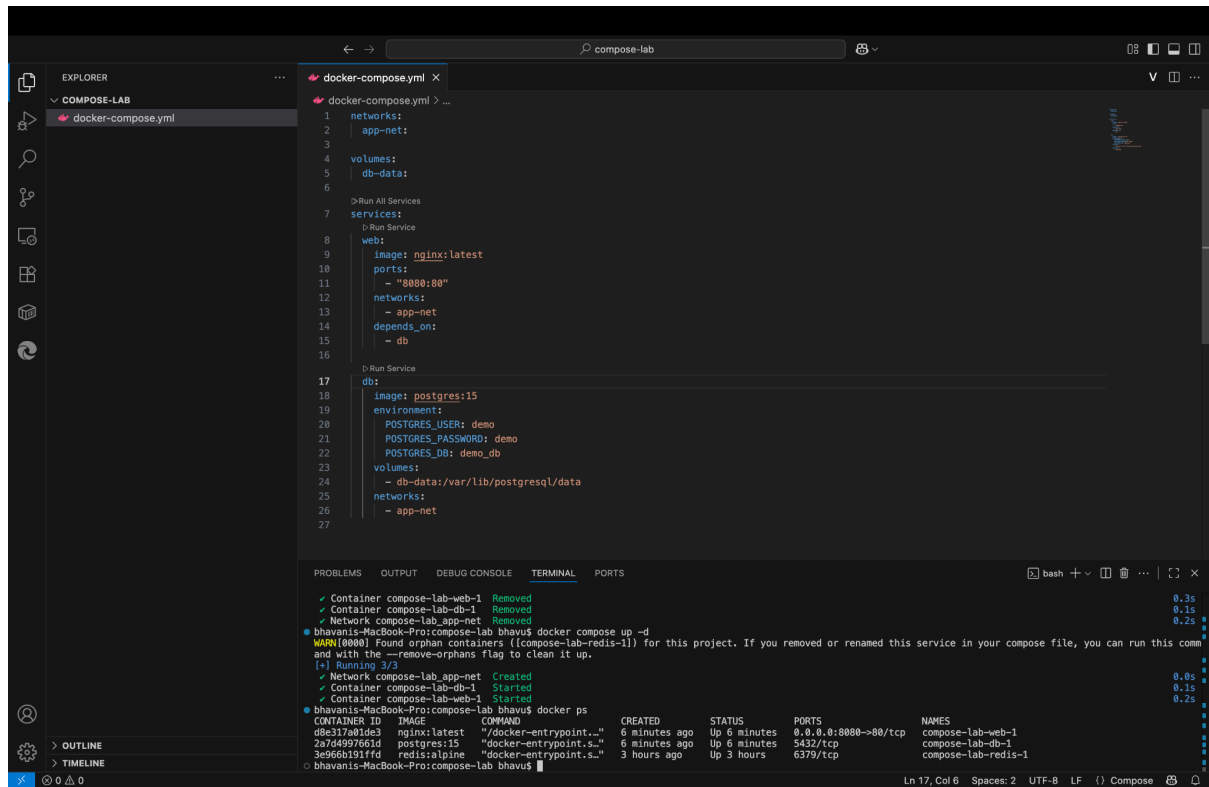
II. Run:

```
docker compose up -d
```

Check that Nginx and Postgres work there as well.

III. Expected Output:

The same services run on the new machine without changes.



4. Networking and persistent storage

Task:

- I. **Update your docker-compose.yml to add a custom network and volume:**

networks:

app-net:

volumes:

db-data:

services:

web:

image: nginx:latest

ports:

- "8080:80"

networks:

- app-net

depends_on:

- db

db:

image: postgres:15

environment:

POSTGRES_USER: demo

POSTGRES_PASSWORD: demo

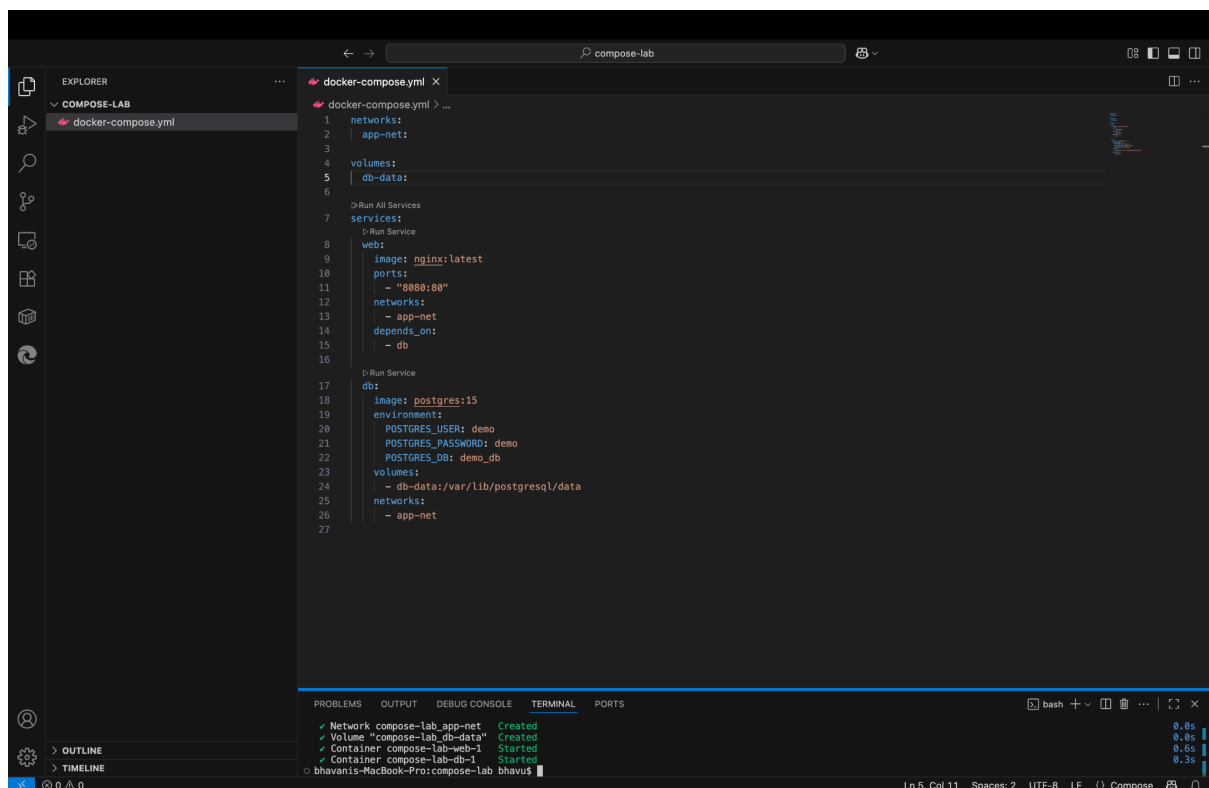
POSTGRES_DB: demo_db

volumes:

- db-data:/var/lib/postgresql/data

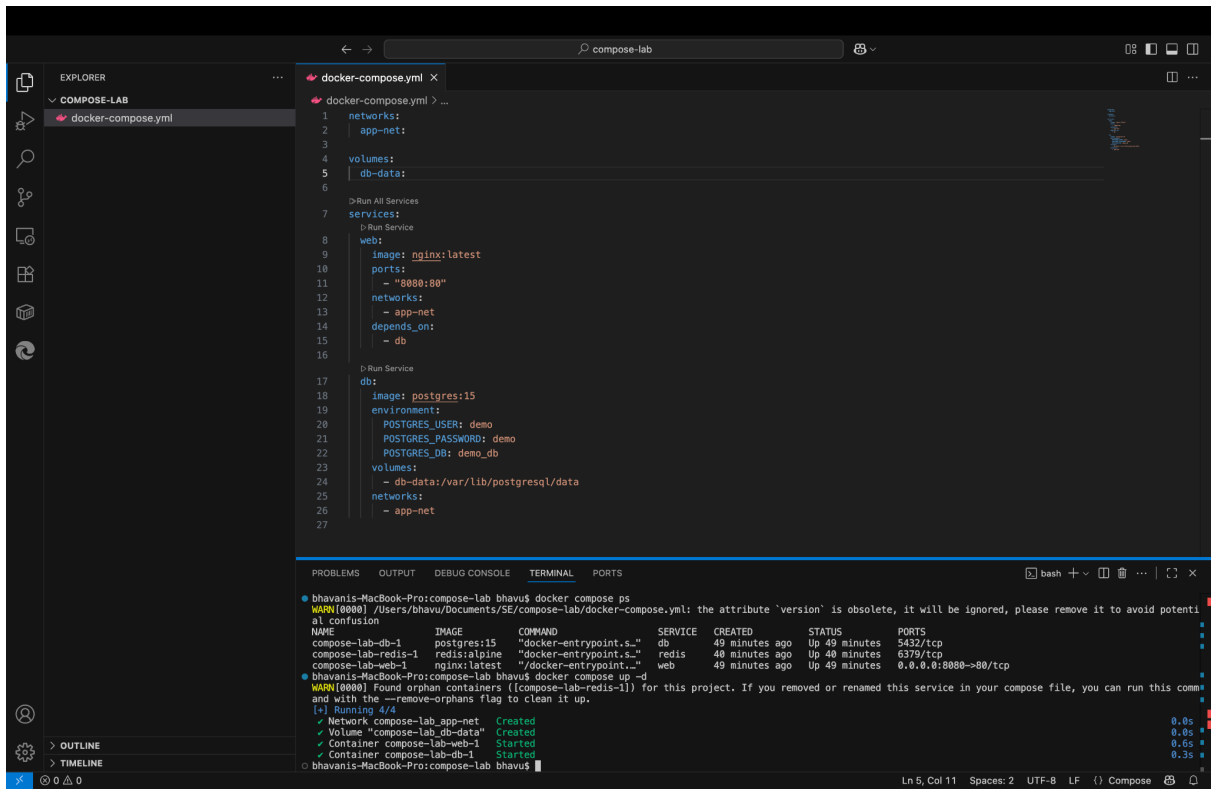
networks:

- app-net



II. Run:

docker compose up -d



III. Insert some data into Postgres (optional with psql).

The screenshot shows a VS Code editor with a file explorer on the left containing a folder named 'COMPOSE-LAB' with a file 'docker-compose.yml'. The main editor displays the content of 'docker-compose.yml', which defines a network 'app-net', a volume 'db-data', and two services: 'web' (nginx:latest) and 'db' (postgres:15). The 'web' service depends on the 'db' service. Below the file editor, the 'TERMINAL' tab is active, showing the output of running 'docker-compose up'. It indicates that containers 'compose-lab-db-1' and 'compose-lab-web-1' are running. The terminal also shows SQL commands being executed in the 'demo_db' container, including creating a table 'users', inserting data for 'Alice' and 'Bob', and a query showing the inserted records.

```
docker-compose.yml
1 networks:
2   app-net:
3
4 volumes:
5   db-data:
6
7 >Run All Services
8 services:
9   web:
10    image: nginx:latest
11    ports:
12      - "8080:80"
13    networks:
14      - app-net
15    depends_on:
16      - db
17   db:
18    image: postgres:15
19    environment:
20      POSTGRES_USER: demo
21      POSTGRES_PASSWORD: demo
```

```
Running 2/3
Container compose-lab-db-1 Running 0.05s
Container compose-lab-web-1 Running 0.05s
bhavanis-MacBook-Pro:compose-lab bhavus$ docker-compose exec db psql -U demo -d demo_db
psql (15.14 (Debian 15.14-1.pgdg15+1))
Type "help" for help.

demo_db=# CREATE TABLE users (
demo_db=# id SERIAL PRIMARY KEY,
demo_db=# name VARCHAR(50),
demo_db=# email VARCHAR(100)
demo_db=# );
demo_db=# INSERT INTO users (name, email) VALUES
demo_db=# ('Alice', 'alice@example.com'),
demo_db=# ('Bob', 'bob@example.com');
demo_db=# CREATE TABLE
demo_db=# INSERT 0 2
demo_db=# SELECT * FROM users;
 id | name | email
----+-----+-----
  1 | Alice | alice@example.com
  2 | Bob  | bob@example.com
(2 rows)

demo_db=#
```

IV. Remove containers:

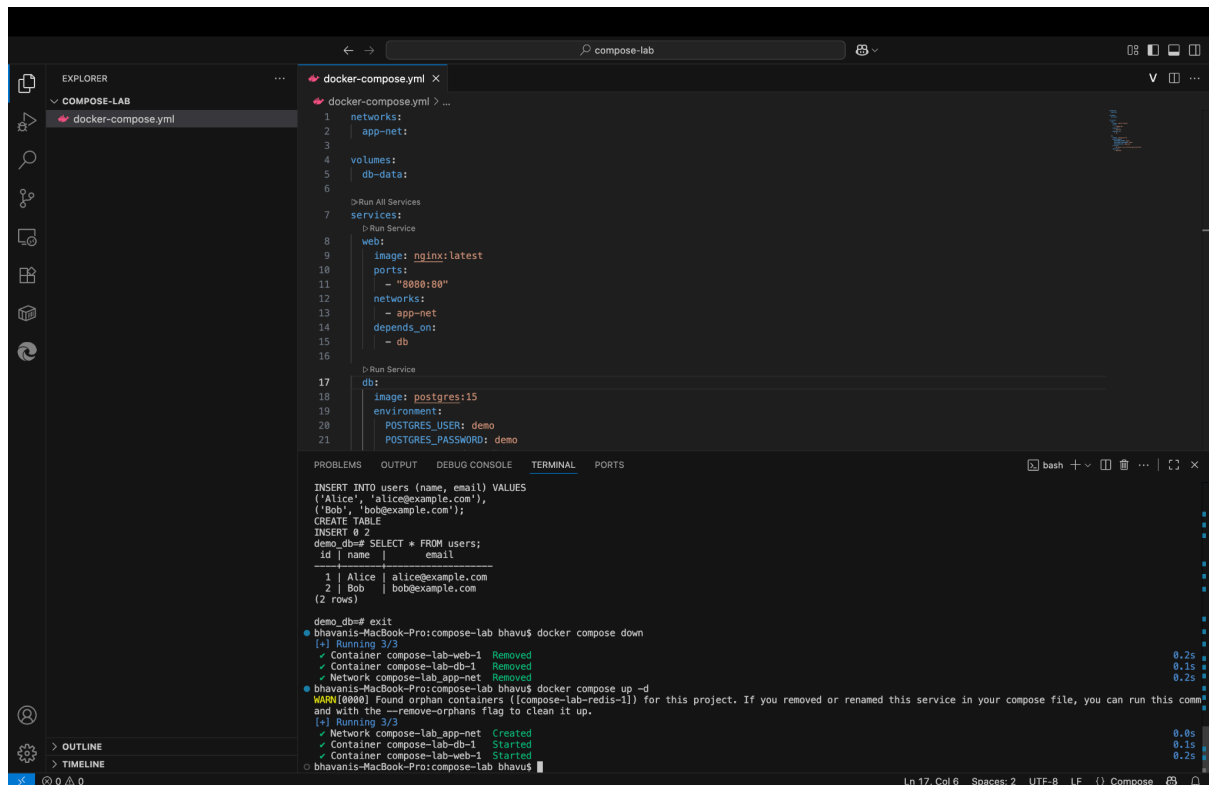
docker compose down

This screenshot shows the same VS Code environment as the previous one, but after running 'docker-compose down'. The 'docker-compose.yml' file is still visible in the editor. The 'TERMINAL' tab now shows the output of the 'down' command, indicating that the containers 'compose-lab-web-1', 'compose-lab-db-1', and the network 'app-net' have been successfully removed. The terminal also shows the user exiting the 'demo_db' container and returning to the shell.

```
demo_db=# exit
bhavanis-MacBook-Pro:compose-lab bhavus$ docker compose down
Running 3/3
Container compose-lab-web-1 Removed 0.25s
Container compose-lab-db-1 Removed 0.15s
Network compose-lab_app-net Removed 0.25s
bhavanis-MacBook-Pro:compose-lab bhavus$
```

V. Start again:

docker compose up -d



The screenshot shows a VS Code editor with a file named `docker-compose.yml` open. The file defines a Docker Compose configuration with two services: `web` and `db`. The `web` service uses the `nginx:latest` image and maps port 8080 to 80. The `db` service uses the `postgres:15` image and sets environment variables for `POSTGRES_USER` and `POSTGRES_PASSWORD` to `demo`. Both services are connected to a network named `app-net`. The `db` service also depends on the `web` service.

The terminal output shows the execution of `docker compose up -d`. It displays the status of the containers: `compose-lab-web-1` is removed, `compose-lab-db-1` is removed, and `compose-lab-app-net` is removed. Then, it shows the creation and starting of the `compose-lab-app-net` network, `compose-lab-db-1` container, and `compose-lab-web-1` container. The output also includes a warning about orphan containers and a table showing the contents of the `demo_db` table.

```
INSERT INTO users (name, email) VALUES
('Alice', 'alice@example.com'),
('Bob', 'bob@example.com');
CREATE TABLE
demo_db=# SELECT * FROM users;
 id | name | email
----+-----+-----
  1 | Alice | alice@example.com
  2 | Bob  | bob@example.com
(2 rows)

demo_db=# exit
bhavanis-MacBook-Pro:compose-lab bhavus$ docker compose down
[+] Running 3/3
 ✓ Container compose-lab-web-1 Removed      0.2s
 ✓ Container compose-lab-db-1 Removed        0.1s
 ✓ Network compose-lab-app-net Removed      0.2s
bhavanis-MacBook-Pro:compose-lab bhavus$ docker compose up -d
WARN[0000] Found orphan containers ([compose-lab-redis-1]) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
[+] Running 3/3
 ✓ Network compose-lab-app-net Created       0.0s
 ✓ Container compose-lab-db-1 Started        0.1s
 ✓ Container compose-lab-web-1 Started       0.2s
bhavanis-MacBook-Pro:compose-lab bhavus$
```

VI. Expected Output:

Database data persists across restarts.

Services communicate via the `app-net` network using service names.

5.Faster iteration during development

Task:

I. Create a simple Flask app in `app.py`:

```
from flask import Flask
```

```
app = Flask(__name__)
```

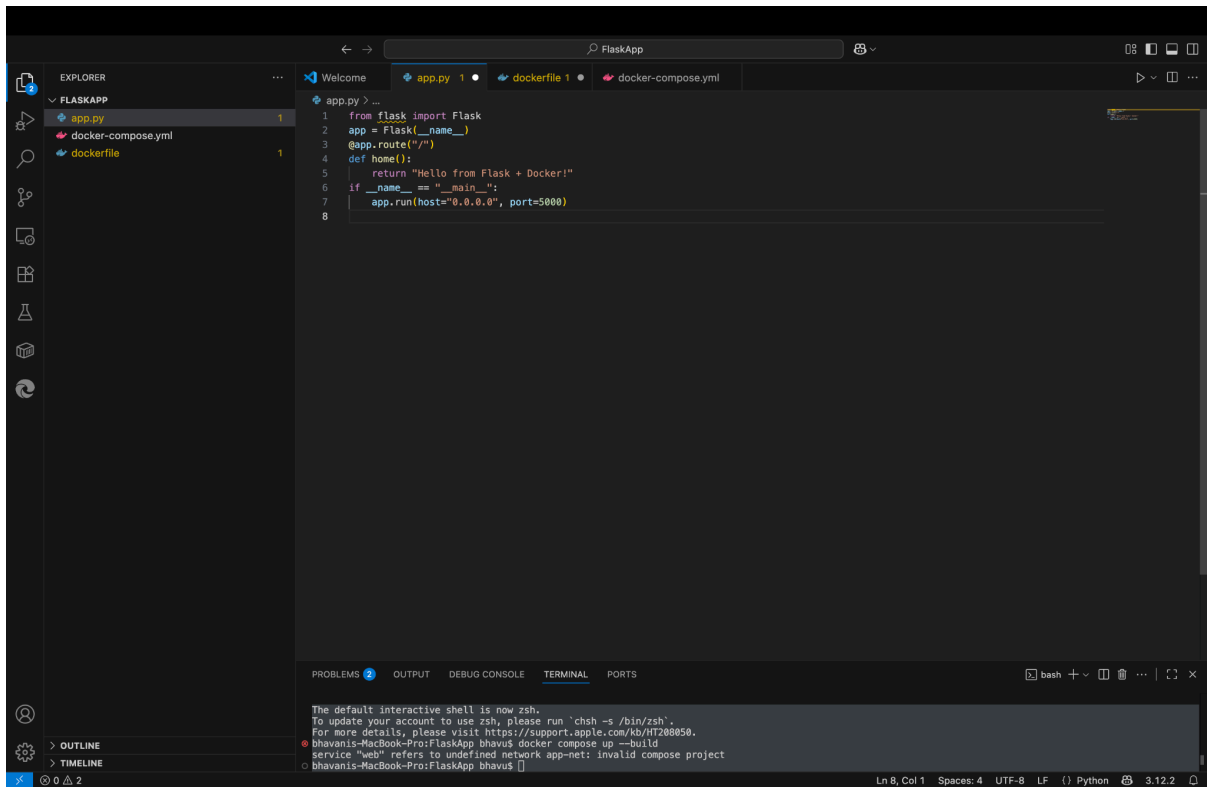
```
@app.route("/")
```

```
def home():
```

```
    return "Hello from Flask + Docker!"
```

```
if __name__ == "__main__":
```

```
    app.run(host="0.0.0.0", port=5000)
```



II. Add a Dockerfile in the same folder:

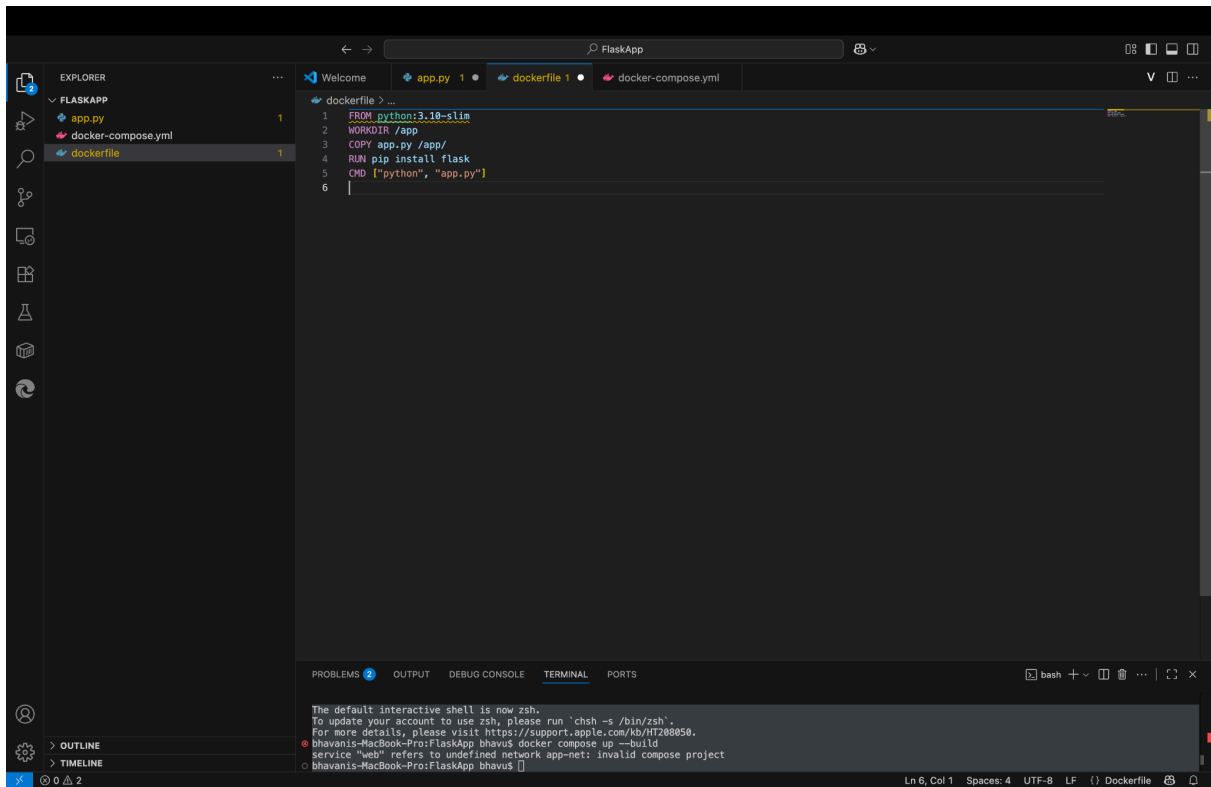
FROM python:3.10-slim

WORKDIR /app

COPY app.py /app/

RUN pip install flask

CMD ["python", "app.py"]



III. Update docker-compose.yml:

web:

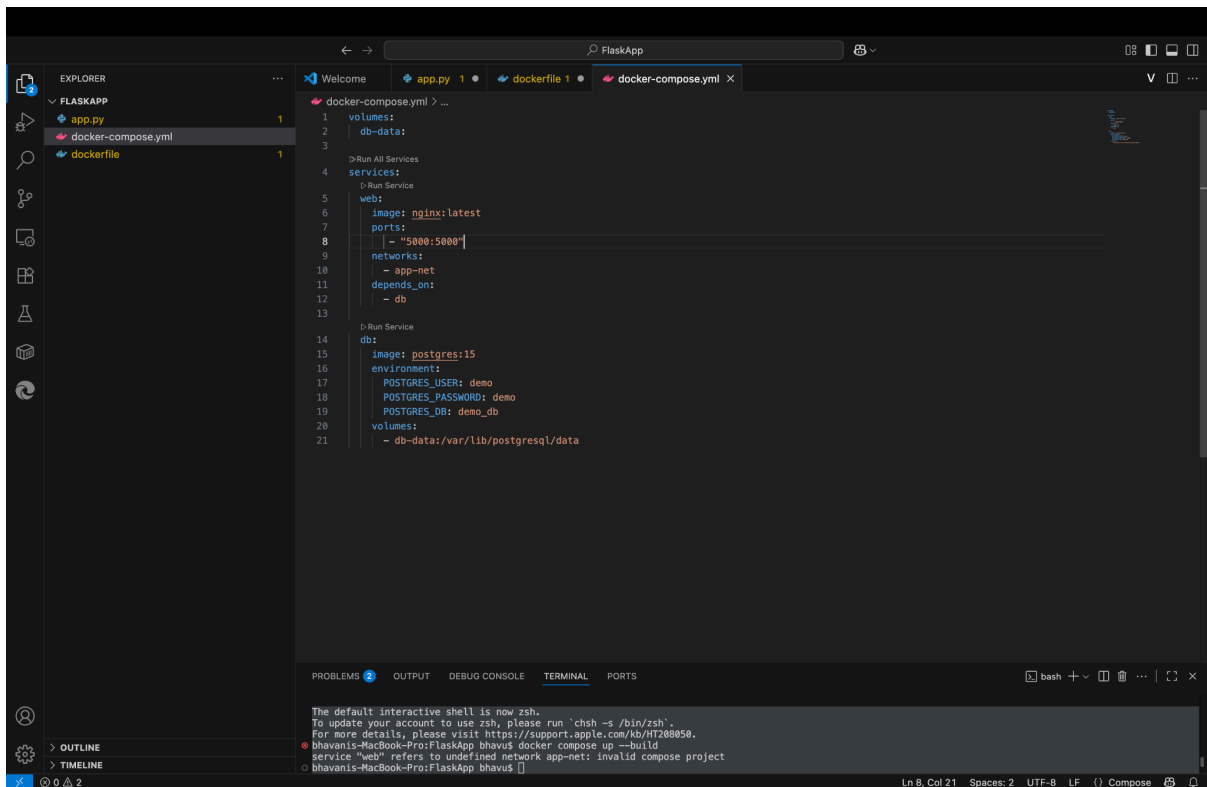
build: .

ports:

- "5000:5000"

depends_on:

- db



```
1 volumes:
2   db-data:
3
4 services:
5   web:
6     image: nginx:latest
7     ports:
8       - "5000:5000"
9     networks:
10      - app-net
11     depends_on:
12       - db
13
14   db:
15     image: postgres:15
16     environment:
17       POSTGRES_USER: demo
18       POSTGRES_PASSWORD: demo
19       POSTGRES_DB: demo_db
20     volumes:
21       - db-data:/var/lib/postgresql/data
```

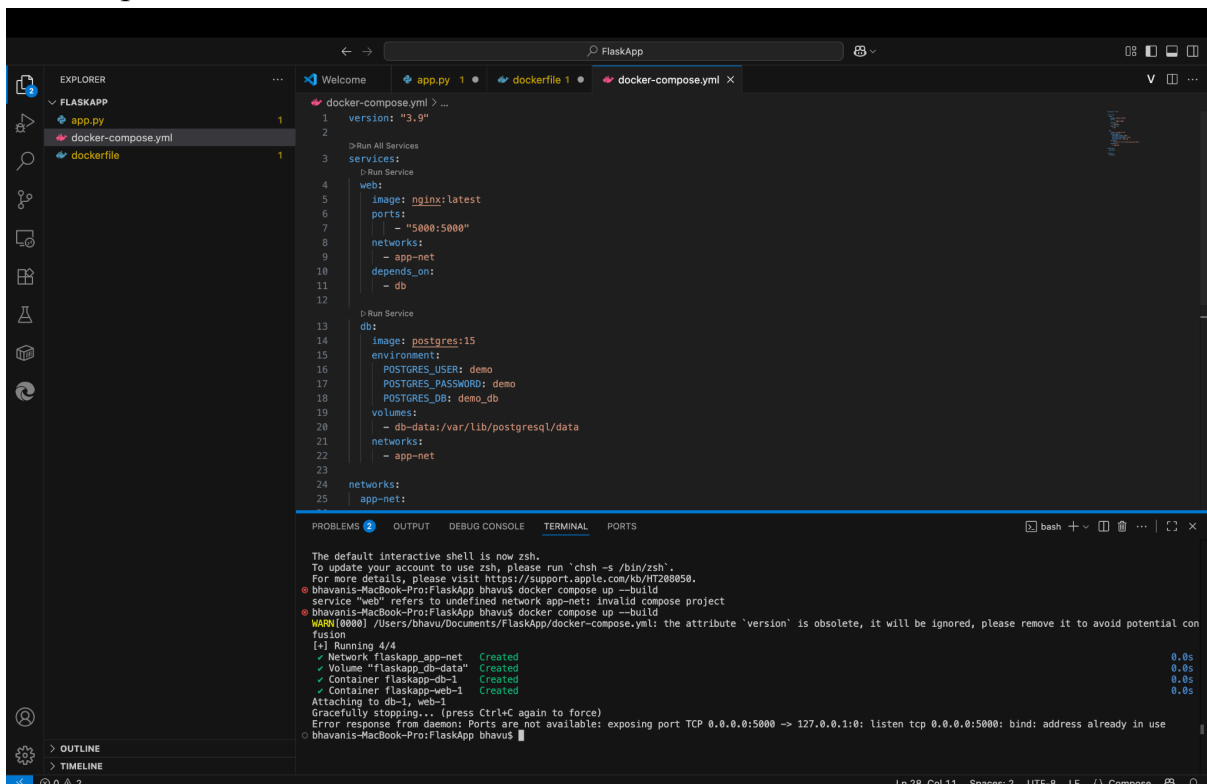
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

The default interactive shell is now zsh.
To update your account to use zsh, please run 'chsh -s /bin/zsh'.
For more details, please visit <https://support.apple.com/kb/HT208050>.
bhavanis-MacBook-Pro:FlaskApp bhavus docker compose up --build
service "web" refers to undefined network app-net: invalid compose project
bhavanis-MacBook-Pro:FlaskApp bhavus

IV. Run:

docker compose up --build

Visit <http://localhost:5000>.



```
1 version: "3.9"
2
3 services:
4   web:
5     image: nginx:latest
6     ports:
7       - "5000:5000"
8     networks:
9       - app-net
10    depends_on:
11      - db
12
13   db:
14     image: postgres:15
15     environment:
16       POSTGRES_USER: demo
17       POSTGRES_PASSWORD: demo
18       POSTGRES_DB: demo_db
19     volumes:
20       - db-data:/var/lib/postgresql/data
21     networks:
22       - app-net
23
24 networks:
25   app-net:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

The default interactive shell is now zsh.
To update your account to use zsh, please run 'chsh -s /bin/zsh'.
For more details, please visit <https://support.apple.com/kb/HT208050>.
bhavanis-MacBook-Pro:FlaskApp bhavus docker compose up --build
service "web" refers to undefined network app-net: invalid compose project
bhavanis-MacBook-Pro:FlaskApp bhavus docker compose up --build
WARN[0000] /Users/bhavu/Documents/FlaskApp/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] Running 4/4
✓ Network flaskapp_app-net Created 0.0s
✓ Volume "flaskapp_db-data" Created 0.0s
✓ Container flaskapp-db-1 Created 0.0s
✓ Container flaskapp-web-1 Created 0.0s
Attaching to db-1, web-1
Gracefully stopping... (press Ctrl+C again to force)
Error response from daemon: Ports are not available: exposing port TCP 0.0.0.0:5000 -> 127.0.0.1:0: listen tcp 0.0.0.0:5000: bind: address already in use
bhavanis-MacBook-Pro:FlaskApp bhavus

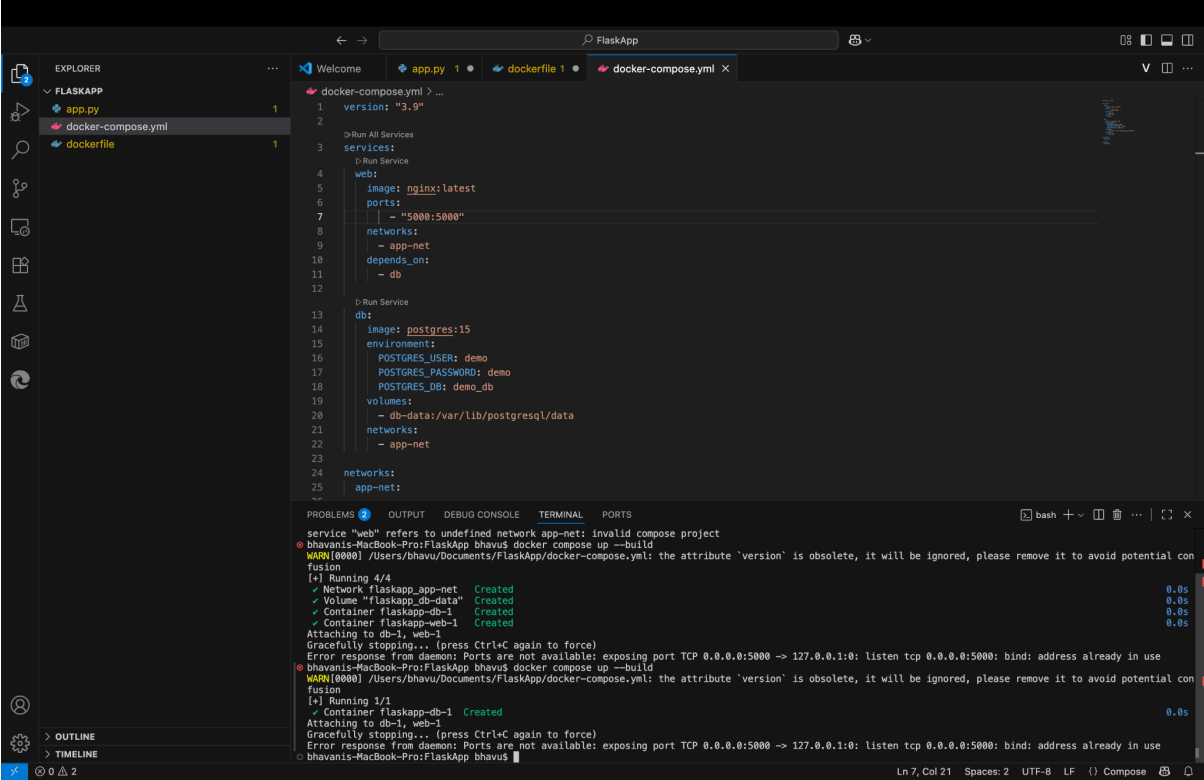
Change the return text in app.py (e.g., "Hello Docker Compose!").

V. Rebuild:

docker compose up --build

VI. Expected Output:

New message appears instantly after rebuild.



The screenshot shows a VS Code editor with a project named 'FlaskApp'. The Explorer sidebar on the left shows files: 'app.py', 'docker-compose.yml', and 'dockerfile'. The main editor area displays the 'docker-compose.yml' file with the following content:

```
1 version: "3.9"
2
3 services:
4   web:
5     image: nginx:latest
6     ports:
7       - "5000:5000"
8     networks:
9       - app-net
10    depends_on:
11      - db
12
13   db:
14     image: postgres:15
15     environment:
16       POSTGRES_USER: demo
17       POSTGRES_PASSWORD: demo
18       POSTGRES_DB: demo_db
19     volumes:
20       - db-data:/var/lib/postgresql/data
21     networks:
22       - app-net
23
24 networks:
25   app-net:
```

Below the editor, the 'TERMINAL' tab shows the output of the command 'docker compose up --build'. The output indicates that the network 'flaskapp_app-net' and volume 'flaskapp_db-data' were created, and containers 'flaskapp-db-1' and 'flaskapp-web-1' were successfully started. A warning message is also visible: 'WARN[0000] /Users/bhavu/Documents/FlaskApp/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential con'. The terminal also shows the process of attaching to the containers and the daemon's response regarding port availability.

Scenario based Questions:

1. You have two applications — a Node.js backend and a Python script that handles scheduled jobs. You're considering running both inside the same container. Should you run multiple applications in the same Docker container?

2. You have a Flask API and an Nginx server. You want to run both containers on the same host and expose them on ports 5000 and 80 respectively. How can you expose both applications from different containers?

3. You want to run a React frontend, an Express.js backend and MongoDB – all together. How do you run and manage all three together?

4. You try to run two containers that both expose port 8080 on the host.

What happens, and how can you run both apps?

5. You updated some code and want to restart your frontend, backend, and DB containers quickly. What is the easiest way to restart all services?
6. Your frontend has a new release, but you don't want to restart the backend or DB. How can you update and rebuild only the frontend container?
7. Tomcat is listening on port 8080, but you get a "Connection Refused" error in the browser. Why?
8. You're not sure which container is using port 3000. How do you check?
9. How do you stop and clean up your running Tomcat container and image?
10. You want to share your app with a teammate. What do you do with the Docker image?

Conclusion:

Docker Compose simplifies multi-container application management by allowing developers to define, configure, and run services in a single YAML file. It streamlines deployment, ensures consistent environments, and reduces manual setup, making it an essential tool for orchestrating complex, container-based applications efficiently.